

## Resumen/tutorial de cómo empezar a crear hilos

La clase `Thread` en Java es la base para la programación concurrente en Java y permite ejecutar múltiples tareas de manera simultánea. Cada instancia de `Thread` representa un hilo de ejecución separado, lo que permite que distintas partes de un programa se ejecuten en paralelo.

Aquí te explico los conceptos y métodos esenciales de la clase `Thread` y cómo se usa en programación concurrente en Java.

### 1. Crear y Ejecutar un Hilo

Hay dos maneras principales de crear un hilo en Java:

- **Extendiendo la clase `Thread`:** Se crea una clase que extiende `Thread` y sobrescribe su método `run()` para definir la tarea que el hilo ejecutará.

```
java
Copiar código
class MiHilo extends Thread {
    public void run() {
        System.out.println("Hilo en ejecución: " +
Thread.currentThread().getName());
    }
}

public class Main {
    public static void main(String[] args) {
        MiHilo hilol = new MiHilo();
        hilol.start(); // Inicia el hilo, que llama a `run()`
    }
}
```

- **Implementando la interfaz `Runnable`:** Otra forma es crear una clase que implemente `Runnable` y pasar una instancia de esta clase al constructor de `Thread`. Esto permite mayor flexibilidad, como heredar de otra clase además de `Thread`.

```
java
Copiar código
class MiRunnable implements Runnable {
    public void run() {
        System.out.println("Hilo en ejecución: " +
Thread.currentThread().getName());
    }
}

public class Main {
    public static void main(String[] args) {
        Thread hilol = new Thread(new MiRunnable());
        hilol.start(); // Inicia el hilo
    }
}
```

// los ejemplos extenderán la clase `thread` en toda la unidad

## 2. Métodos Importantes de la Clase `Thread`

| Método                                 | Descripción                                                                                                 |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>start()</code>                   | Inicia el hilo, haciendo que llame al método <code>run()</code> .                                           |
| <code>run()</code>                     | Contiene el código que se ejecuta en el hilo. Debe sobrescribirse cuando extendemos <code>Thread</code> .   |
| <code>sleep(long millis)</code>        | Hace que el hilo se detenga temporalmente durante el tiempo especificado en milisegundos.                   |
| <code>join()</code>                    | Espera a que el hilo termine antes de que el programa continúe con el siguiente bloque de código.           |
| <code>interrupt()</code>               | Interrumpe el hilo. Puede utilizarse para detenerlo antes de que termine.                                   |
| <code>isAlive()</code>                 | Verifica si el hilo está activo o en ejecución.                                                             |
| <code>setPriority(int priority)</code> | Establece la prioridad del hilo, con valores de 1 (mínima) a 10 (máxima). La prioridad predeterminada es 5. |
| <code>getPriority()</code>             | Devuelve la prioridad del hilo.                                                                             |
| <code>getName()</code>                 | Obtiene el nombre del hilo.                                                                                 |
| <code>setName(String name)</code>      | Asigna un nombre al hilo.                                                                                   |

## 3. Ciclo de Vida de un Hilo

Un hilo en Java pasa por diferentes estados a lo largo de su ciclo de vida:

- **Nuevo (NEW):** Cuando se crea el hilo, pero aún no se ha iniciado.
- **Ejecutable (RUNNABLE):** Después de llamar a `start()`, el hilo está listo para ejecutarse.
- **Bloqueado (BLOCKED):** Cuando el hilo intenta acceder a un recurso que ya está siendo usado por otro hilo.
- **En espera (WAITING) o En espera con tiempo (TIMED\_WAITING):** Cuando el hilo está esperando indefinidamente (`wait()`) o durante un período de tiempo específico (`sleep()`, `join()` con un tiempo).
- **Terminado (TERMINATED):** El hilo ha completado la ejecución de `run()` y ha finalizado.

## 4. Conceptos Avanzados en Programación de Hilos

- **Sincronización:** La palabra clave `synchronized` asegura que solo un hilo acceda a una sección crítica del código a la vez, evitando condiciones de carrera.
- **`wait()`, `notify()`, `notifyAll()`:** Métodos para la comunicación entre hilos, permitiendo que un hilo espere hasta que otro lo despierte.
- **Semáforos (Semaphore):** Permiten controlar el acceso a recursos compartidos para varios hilos simultáneamente.
- **Interrupciones:** Permiten detener o pausar hilos de forma segura mediante `interrupt()`, y los hilos deben verificar regularmente si fueron interrumpidos.

Ahora veamos las diapositivas para entender ejemplos más avanzados: